

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: Database Management Systems
COURSE CODE: CSA05

COURSE OUTCOME COVERED

CO3: *Analyze relational databases using functional dependencies, normalization (1NF to 5NF), and key constraints to identify redundancy and ensure data integrity. (BL2, BL3)*

Activity Tool : Experiments (High)
Assessment Tool Weightage: 40%

QUESTIONS		Total Marks: 50	
Q.No	Question	Bloom's Level	Marks
1	For the relation STUDENT_COURSE(SID, SName, CID, CName, Instructor, Grade), identify all functional dependencies and determine the candidate keys.	BL2 – Understand	5
2	Demonstrate the process of converting an unnormalized relation into 1NF with step-by-step justification and example.	BL3 – Apply	5
3	Given relation R(A,B,C,D,E) with FDs: $A \rightarrow BC$, $BC \rightarrow D$, $D \rightarrow E$. Analyze and decompose it to 2NF eliminating partial dependencies.	BL3 – Apply	5
4	Apply 3NF decomposition on the relation EMPLOYEE(EmpID, EmpName, DeptID, DeptName, ManagerID) with transitive dependencies.	BL3 – Apply	5
5	Verify whether R(A,B,C,D) with FDs: $AB \rightarrow C$, $C \rightarrow D$, $D \rightarrow A$ is in BCNF. If not, decompose it into BCNF.	BL3 – Apply	5
6	Experimentally verify lossless-join decomposition for a given relation using the tabular (Chase) algorithm.	BL3 – Apply	5
7	Identify the multi-valued dependencies in TEACHER(TID, Subject, Hobby) and decompose it into 4NF.	BL3 – Apply	5
8	Demonstrate how join dependencies lead to redundancy. Decompose a given relation to 5NF (Project-Join Normal Form).	BL3 – Apply	5
9	Given a poorly designed database schema for a college, identify all anomalies	BL2 –	5

	(insertion, deletion, update) and normalize it to 3NF.	Understand	
10	Compute the closure of attribute sets for the given FDs and determine all candidate keys for the relation.	BL3 – Apply	5

Code of Conduct:

*I Shaik Ayesha Tabassum(192525074) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student: S.Ayesha Tabassum

RUBRICS FOR CALCULATION:

Criteria	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	Demonstrates thorough understanding and effectively integrates multiple concepts/technologies	Good understanding with proper integration of most concepts	Basic understanding; limited or partial integration	Poor understanding; unable to integrate concepts
Experimental Design	Well-planned, systematic implementation with optimal solution	Correct implementation with minor issues	Basic implementation with errors or inefficiencies	Incorrect or incomplete implementation
Use of Tools	Effective and advanced use of tools/technologies with proper configuration	Appropriate use of tools with correct execution	Limited or inconsistent use of tools	Incorrect or minimal use of tools
Analysis of Results	Insightful analysis with accurate interpretation and validation of results	Good analysis with reasonable interpretation	Basic analysis with limited insights	Poor or incorrect analysis of results
Documentation	Clear, well-structured documentation with diagrams, code, and results	Organized documentation with necessary details	Basic documentation with missing details	Poor or incomplete documentation

1. For the relation STUDENT_COURSE(SID, SName, CID, CName, Instructor, Grade), identify all functional dependencies and determine the candidate keys.

Functional Dependencies and Candidate Key

Consider the relation:

STUDENT_COURSE(SID, SName, CID, CName, Instructor, Grade)

This relation stores information about students, courses, instructors, and the grades obtained by students.

1. Meaning of Attributes

- **SID** – Student ID, which uniquely identifies a student.
- **SName** – Name of the student.
- **CID** – Course ID, which uniquely identifies a course.
- **CName** – Name of the course.
- **Instructor** – Instructor teaching the course.
- **Grade** – Grade obtained by the student in that course.

2. Functional Dependencies

A functional dependency means that one attribute or a group of attributes determines another attribute.

The functional dependencies are:

1. **SID** → **SName**

A particular student ID identifies only one student name.

2. **CID** → **CName**

A particular course ID identifies only one course name.

3. **CID** → **Instructor**

A course ID determines the instructor assigned to that course.

4. **(SID, CID)** → **Grade**

The grade depends on both the student and the course. A student can take multiple courses, so SID alone cannot determine the grade. Similarly, CID alone cannot determine the grade.

Therefore, the functional dependencies are:

$SID \rightarrow SName$ $CID \rightarrow CName$ $CID \rightarrow Instructor$ $(SID, CID) \rightarrow Grade$

3. Finding the Candidate Key

To find the candidate key, we calculate the **attribute closure**.

Check SID

Starting with SID:

$SID^+ = \{SID, SName\}$

SID can determine only SName. It cannot determine CID, CName, Instructor, and Grade.

Therefore, **SID is not a candidate key**.

Check CID

Starting with CID:

$CID^+ = \{CID, CName, Instructor\}$

CID can determine CName and Instructor, but it cannot determine SID, SName, and Grade.

Therefore, **CID is not a candidate key**.

Check (SID, CID)

Now consider:

$(SID, CID)^+$

Using the functional dependencies:

- $SID \rightarrow SName$
- $CID \rightarrow CName$
- $CID \rightarrow Instructor$
- $(SID, CID) \rightarrow Grade$

We get:

$(SID, CID)^+ = \{SID, SName, CID, CName, Instructor, Grade\}$

The closure contains **all attributes** in the relation.

Hence:

(SID,CID)

is a **candidate key**.

4. Final Result

Functional Dependency Explanation

SID → SName	Student ID determines student name
CID → CName	Course ID determines course name
CID → Instructor	Course ID determines instructor
(SID, CID) → Grade	Student and course together determine grade

Conclusion

The relation **STUDENT_COURSE** has **(SID, CID)** as its candidate key because a student can enroll in multiple courses and a course can contain multiple students. Therefore, neither SID nor CID alone can uniquely identify a record. The combination of **SID and CID** uniquely identifies a student's enrollment in a particular course and determines the corresponding grade. **Candidate Key = (SID, CID)**.

2. Demonstrate the process of converting an unnormalized relation into 1NF with step-by-step justification and example.

Converting an Unnormalized Relation into 1NF

Introduction

First Normal Form (1NF) is the first step in database normalization. A relation is said to be in 1NF when:

- Each column contains **atomic (single) values**.
- There are **no repeating groups** or multivalued attributes.
- Each row represents a single record.
- Each column contains values of the same type.

Let us understand the conversion using a **Student-Course** example.

Step 1: Consider an Unnormalized Relation

Suppose we have the following relation:

STUDENT(SID, SName, Courses)

SID	SName	Courses
101	Arun	DBMS, Java, Python
102	Priya	C, DBMS
103	Ravi	Java, Python, Cloud

Here, the **Courses** attribute contains multiple values in a single cell.

For example:

101 | Arun | DBMS, Java, Python

The value **DBMS, Java, Python** is not atomic.

Therefore, the relation is **not in 1NF**.

Step 2: Identify the Problem

The Courses attribute is a **multivalued attribute**.

For student 101:

- DBMS
- Java
- Python

are stored together in one cell.

This creates problems when we want to:

- Search for students taking a particular course.
- Add a new course for a student.
- Delete one course without affecting other courses.
- Update course information.

Therefore, we need to remove the repeating group.

Step 3: Remove the Repeating Group

We create a separate row for each course taken by a student.

The relation becomes:

STUDENT_COURSE(SID, SName, Course)

SID	SName	Course
101	Arun	DBMS
101	Arun	Java
101	Arun	Python
102	Priya	C
102	Priya	DBMS
103	Ravi	Java
103	Ravi	Python
103	Ravi	Cloud

Now every cell contains **only one value**.

For example:

Instead of:

DBMS, Java, Python

we have three separate records:

DBMS

Java

Python

Step 4: Identify the Key

Since one student can take many courses, **SID alone cannot uniquely identify a row.**

For example:

SID	SName	Course
101	Arun	DBMS
101	Arun	Java

SID 101 occurs more than once.

Therefore, the combination of **SID and Course** can be used as the key.

(SID, Course)

This combination uniquely identifies each student-course record.

Step 5: Verify 1NF

The resulting relation is:

STUDENT_COURSE(SID, SName, Course)

It satisfies 1NF because:

1. Each attribute contains a **single atomic value**.
2. There are **no multivalued attributes**.
3. There are **no repeating groups**.
4. Each row represents one student taking one course.
5. (SID, Course) uniquely identifies each record.

Therefore:

STUDENT_COURSE is in 1NF

Before and After

Unnormalized:

SID	SName	Courses
101	Arun	DBMS, Java, Python
102	Priya	C, DBMS

After converting to 1NF:

SID	SName	Course
101	Arun	DBMS
101	Arun	Java
101	Arun	Python
102	Priya	C
102	Priya	DBMS

Conclusion

The conversion from an unnormalized relation to **1NF** mainly involves **removing repeating groups and converting multivalued attributes into atomic values**. In the example, the Courses attribute originally contained multiple course names in one cell. By creating a separate row for each student-course combination, all values become atomic and the relation satisfies **First Normal Form (1NF)**.

3. Given relation $R(A,B,C,D,E)$ with FDs: $A \rightarrow BC$, $BC \rightarrow D$, $D \rightarrow E$. Analyze and decompose it to 2NF eliminating partial dependencies.

Decomposition of Relation $R(A, B, C, D, E)$ into 2NF

Given the relation:

$R(A,B,C,D,E)$

Functional Dependencies (FDs):

$A \rightarrow BC$ $BC \rightarrow D$ $D \rightarrow E$

The objective is to **analyze the functional dependencies and decompose the relation into 2NF by eliminating partial dependencies.**

Step 1: Find the Candidate Key

We first calculate the closure of A .

Starting with:

$A^+ = \{A\}$

Using:

$A \rightarrow BC$

we get:

$A^+ = \{A,B,C\}$

Using:

$BC \rightarrow D$

we get:

$A^+ = \{A,B,C,D\}$

Using:

$D \rightarrow E$

we get:

$A^+ = \{A, B, C, D, E\}$

Therefore:

$A^+ = \{A, B, C, D, E\}$

Since A determines all attributes, **A is a candidate key.**

Candidate Key=A

Step 2: Check for Partial Dependencies

A **partial dependency** occurs when a **non-key attribute depends on only part of a composite candidate key.**

Here, the candidate key is only **A**, which is a **single attribute.**

Therefore, there cannot be a partial dependency because there is no composite key with multiple attributes.

For example:

$A \rightarrow B, C$

is a dependency from the **complete key A**, not a partial dependency.

Thus:

There are no partial dependencies

Therefore, **R is already in 2NF.**

Step 3: But There Is a Transitive Dependency

Although there is no partial dependency, there is a **transitive dependency:**

$A \rightarrow BC$

and

$BC \rightarrow D$

and

$D \rightarrow E$

Therefore:

$A \rightarrow D$

and:

$A \rightarrow E$

The dependency chain is:

$A \rightarrow BC \rightarrow D \rightarrow E$

Here, **D is a non-key attribute**, and D determines E.

So:

$D \rightarrow E$

is a **transitive dependency**.

This means the relation is **not in 3NF**, although it is already in **2NF**.

Step 4: Decomposition

Since the question specifically asks for **2NF**, no decomposition is actually required to eliminate partial dependencies.

Therefore:

$R(A,B,C,D,E)$ is already in 2NF

However, if we want to remove the transitive dependency and move toward **3NF**, we can decompose it as:

Relation 1

$R_1(A,B,C)$

with:

$A \rightarrow BC$

Relation 2

$R_2(B,C,D)$

with:

$BC \rightarrow D$

Relation 3

$R_3(D, E)$

with:

$D \rightarrow E$

So the decomposition is:

$R_1(A, B, C), R_2(B, C, D), R_3(D, E)$

Final Answer

Property	Result
Relation	$R(A, B, C, D, E)$
FDs	$A \rightarrow BC, BC \rightarrow D, D \rightarrow E$
Candidate Key	A
Partial Dependency	None
2NF	Already satisfied
Transitive Dependency	$D \rightarrow E$
3NF decomposition	$R_1(A, B, C), R_2(B, C, D), R_3(D, E)$

Conclusion

The important point is that **2NF only eliminates partial dependencies**. Since the candidate key is the single attribute **A**, partial dependency is impossible. Hence, the original relation **$R(A, B, C, D, E)$ is already in 2NF**. The dependency **$D \rightarrow E$** is a transitive dependency, which needs to be removed when converting the relation to **3NF**, not 2NF.

4. Apply 3NF decomposition on the relation EMPLOYEE(EmpID, EmpName, DeptID, DeptName, ManagerID) with transitive dependencies.

3NF Decomposition of EMPLOYEE Relation

```
mysql> CREATE DATABASE College;
Query OK, 1 row affected (0.01 sec)

mysql> USE College;
Database changed
mysql> CREATE TABLE EMPLOYEE_OLD (
  ->   EmpID INT,
  ->   EmpName VARCHAR(50),
  ->   DeptID INT,
  ->   DeptName VARCHAR(50),
  ->   ManagerID INT
  -> );
Query OK, 0 rows affected (0.06 sec)

mysql> INSERT INTO EMPLOYEE_OLD
  -> VALUES
  -> (101, 'Arun', 10, 'CSE', 501),
  -> (102, 'Priya', 10, 'CSE', 501),
  -> (103, 'Ravi', 20, 'ECE', 502),
  -> (104, 'Meena', 20, 'ECE', 502);
Query OK, 4 rows affected (0.01 sec)
Records: 4 Duplicates: 0 Warnings: 0

mysql> SELECT * FROM EMPLOYEE_OLD;
+-----+-----+-----+-----+-----+
| EmpID | EmpName | DeptID | DeptName | ManagerID |
+-----+-----+-----+-----+-----+
| 101   | Arun    | 10     | CSE      | 501        |
| 102   | Priya   | 10     | CSE      | 501        |
+-----+-----+-----+-----+-----+
```

```
mysql> SELECT * FROM EMPLOYEE_OLD;
+-----+-----+-----+-----+-----+
| EmpID | EmpName | DeptID | DeptName | ManagerID |
+-----+-----+-----+-----+-----+
| 101   | Arun    | 10     | CSE      | 501        |
| 102   | Priya   | 10     | CSE      | 501        |
| 103   | Ravi    | 20     | ECE      | 502        |
| 104   | Meena   | 20     | ECE      | 502        |
+-----+-----+-----+-----+-----+
4 rows in set (0.00 sec)

mysql> CREATE TABLE DEPARTMENT (
  ->   DeptID INT PRIMARY KEY,
  ->   DeptName VARCHAR(50),
  ->   ManagerID INT
  -> );
Query OK, 0 rows affected (0.03 sec)

mysql> INSERT INTO DEPARTMENT
  -> VALUES
  -> (10, 'CSE', 501),
  -> (20, 'ECE', 502);
Query OK, 2 rows affected (0.01 sec)
Records: 2 Duplicates: 0 Warnings: 0
```

```
mysql> SELECT * FROM DEPARTMENT;
+-----+-----+-----+
| DeptID | DeptName | ManagerID |
+-----+-----+-----+
| 10     | CSE      | 501        |
| 20     | ECE      | 502        |
+-----+-----+-----+
2 rows in set (0.00 sec)

mysql> CREATE TABLE EMPLOYEE (
  ->   EmpID INT PRIMARY KEY,
  ->   EmpName VARCHAR(50),
  ->   DeptID INT,
  ->   FOREIGN KEY (DeptID) REFERENCES DEPARTMENT(DeptID)
  -> );
Query OK, 0 rows affected (0.09 sec)

mysql> INSERT INTO EMPLOYEE
  -> VALUES
  -> (101, 'Arun', 10),
  -> (102, 'Priya', 10),
  -> (103, 'Ravi', 20),
  -> (104, 'Meena', 20);
Query OK, 4 rows affected (0.01 sec)
Records: 4 Duplicates: 0 Warnings: 0
```

```
mysql> SELECT * FROM EMPLOYEE;
+-----+-----+-----+
| EmpID | EmpName | DeptID |
+-----+-----+-----+
| 101   | Arun    | 10     |
| 102   | Priya   | 10     |
| 103   | Ravi    | 20     |
| 104   | Meena   | 20     |
+-----+-----+-----+
4 rows in set (0.00 sec)
```

```
mysql> SELECT
-> E.EmpID,
-> E.EmpName,
-> E.DeptID,
-> D.DeptName,
-> D.ManagerID
-> FROM EMPLOYEE E
-> JOIN DEPARTMENT D
-> ON E.DeptID = D.DeptID;
+-----+-----+-----+-----+-----+
| EmpID | EmpName | DeptID | DeptName | ManagerID |
+-----+-----+-----+-----+-----+
| 101   | Arun    | 10     | CSE      | 501       |
| 102   | Priya   | 10     | CSE      | 501       |
| 103   | Ravi    | 20     | ECE      | 502       |
| 104   | Meena   | 20     | ECE      | 502       |
+-----+-----+-----+-----+-----+
4 rows in set (0.01 sec)
```

```
mysql> UPDATE DEPARTMENT
-> SET DeptName = 'Computer Science'
-> WHERE DeptID = 10;
Query OK, 1 row affected (0.01 sec)
Rows matched: 1  Changed: 1  Warnings: 0
```

```
mysql> SELECT * FROM DEPARTMENT;
+-----+-----+-----+
| DeptID | DeptName          | ManagerID |
+-----+-----+-----+
| 10     | Computer Science  | 501       |
| 20     | ECE               | 502       |
+-----+-----+-----+
2 rows in set (0.00 sec)
```

Final Decomposition

EMPLOYEE(EmpID,EmpName,DeptID) DEPARTMENT(DeptID,DeptName,ManagerID)

DeptID in the **EMPLOYEE** table acts as a **foreign key** referencing DEPARTMENT(DeptID).

Conclusion

The original EMPLOYEE relation contains a transitive dependency:

$EmpID \rightarrow DeptID \rightarrow DeptName, ManagerID$

By separating department information into a **DEPARTMENT** relation, the transitive dependency is removed. The resulting relations are in **3NF**, reducing data redundancy and preventing update, insertion, and deletion anomalies.

5. Verify whether $R(A,B,C,D)$ with FDs: $AB \rightarrow C$, $C \rightarrow D$, $D \rightarrow A$ is in BCNF. If not, decompose it into BCNF

BCNF Verification and Decomposition

Consider the relation:

$R(A,B,C,D)$

with the functional dependencies:

$AB \rightarrow C$ $C \rightarrow D$ $D \rightarrow A$

We need to check whether the relation satisfies **Boyce-Codd Normal Form (BCNF)** and, if not, decompose it.

Step 1: Find the Candidate Keys

First, find the closure of **AB**:

$(AB)^+ = \{A, B\}$

Using:

$AB \rightarrow C$

we get:

$(AB)^+ = \{A, B, C\}$

Using:

$C \rightarrow D$

we get:

$(AB)^+ = \{A, B, C, D\}$

Therefore:

AB is a candidate key

Now check **BC**:

$(BC)^+ = \{B, C\}$

Using:

$$C \rightarrow D$$

we get D.

Using:

$$D \rightarrow A$$

we get A.

Therefore:

$$(BC)^+ = \{A, B, C, D\}$$

So:

BC is a candidate key

Check **BD**:

$$(BD)^+ = \{B, D\}$$

Using:

$$D \rightarrow A$$

we get A.

But we cannot derive C because $AB \rightarrow C$ requires both A and B, which we now have:

$$A, B \rightarrow C$$

Therefore:

$$(BD)^+ = \{A, B, C, D\}$$

So:

BD is a candidate key

Step 2: Check BCNF

A relation is in **BCNF** if, for every non-trivial functional dependency:

$$X \rightarrow Y$$

X must be a superkey.

Now examine each FD.

FD 1: $AB \rightarrow C$

AB is a candidate key.

Therefore, **$AB \rightarrow C$ satisfies BCNF.**

FD 2: $C \rightarrow D$

We found that:

$$C^+ = \{C, D, A\}$$

It does **not** contain B.

Therefore, C is **not a superkey.**

So:

$C \rightarrow D$ violates BCNF

FD 3: $D \rightarrow A$

D is not a superkey because:

$$D^+ = \{D, A\}$$

It does not determine B and C.

Therefore:

$D \rightarrow A$ also violates BCNF

Hence:

R(A,B,C,D) is NOT in BCNF

Step 3: Decompose Using $C \rightarrow D$

Choose the violating dependency:

$$C \rightarrow D$$

Create two relations:

$$R_1(C,D)$$

and

$$R_2(A,B,C)$$

Relation $R_1(C,D)$

$$R_1(C,D)$$

FD:

$$C \rightarrow D$$

Here, C is a key of R_1 because:

$$C^+ = \{C, D\}$$

Therefore, **R_1 is in BCNF.**

Relation $R_2(A,B,C)$

$$R_2(A,B,C)$$

The relevant FD is:

$$AB \rightarrow C$$

Here, AB is a key of R_2 .

Therefore:

R_2 is in BCNF

Final BCNF Decomposition

$R_1(C,D)$ $R_2(A,B,C)$

Verification

Relation	Functional Dependency	Determinant	BCNF?
$R_1(C,D)$	$C \rightarrow D$	C is a key	Yes
$R_2(A,B,C)$	$AB \rightarrow C$	AB is a key	Yes

Therefore, the final decomposition is:

$R_1(C,D)$, $R_2(A,B,C)$

and both relations satisfy **BCNF**.

Conclusion

The original relation **$R(A,B,C,D)$** is **not in BCNF** because $C \rightarrow D$ and $D \rightarrow A$ have determinants that are not superkeys. By decomposing the relation using the violating dependency $C \rightarrow D$, we obtain:

$R_1(C,D)$ and $R_2(A,B,C)$

Both resulting relations satisfy **BCNF**.

.

6. Experimentally verify lossless-join decomposition for a given relation using the tabular (Chase) algorithm.

Experimental Verification of Lossless-Join Decomposition Using Chase Algorithm

1. Introduction

A decomposition of a relation is called **lossless-join** if joining the decomposed relations produces exactly the original relation without adding any spurious tuples.

The **Chase algorithm** is a systematic method used to verify whether a decomposition is lossless with respect to a given set of functional dependencies.

2. Given Relation

Consider:

$R(A,B,C)$

with the functional dependency:

$A \rightarrow B$

Suppose we decompose R into:

$R_1(A,B)$

and

$R_2(A,C)$

We need to experimentally verify whether this decomposition is **lossless**.

3. Create the Chase Table

The original relation has three attributes:

Row	A	B	C
Initial row	a	b	c

For the decomposition, we create a tableau with one row for each decomposed relation.

Initial Table

For $R_1(A,B)$:

- A and B belong to R_1 , so use the original symbols a and b.
- C is not present in R_1 , so use a fresh symbol c_1 .

For $R_2(A,C)$:

- A and C belong to R_2 , so use a and c.
- B is not present in R_2 , so use a fresh symbol b_1 .

Therefore:

Row	A	B	C
R_1	a	b	c_1
R_2	a	b_1	c

4. Apply the Functional Dependency

Given:

$A \rightarrow B$

The two rows have the same value for **A**:

Row	A
R_1	a
R_2	a

According to the FD:

$A \rightarrow B$

if two rows have the same A value, they must have the same B value.

Therefore, we replace b_1 with b.

The tableau becomes:

Row	A	B	C
R_1	a	b	c_1
R_2	a	b	c

5. Check the Lossless-Join Condition

We check whether any row becomes:

(a,b,c)

That is, a row containing the **original symbols** in all columns.

Current rows are:

- $R_1 = (a, b, c_1)$
- $R_2 = (a, b, c)$

The second row has:

(a,b,c)

which is exactly the distinguished original row.

Therefore, the chase succeeds.

6. Conclusion

Since the tableau contains a row consisting entirely of the original symbols:

(a,b,c)

the decomposition:

$R(A,B,C) \rightarrow R_1(A,B), R_2(A,C)$

is **lossless-join** with respect to the functional dependency:

$A \rightarrow B$

Final Result

Decomposition is LOSSLESS

The natural join of $R_1(A,B)$ and $R_2(A,C)$ can reconstruct the original relation without generating spurious tuples.

7. Identify the multi-valued dependencies in TEACHER(TID, Subject, Hobby) and decompose it into 4NF.

Multi-Valued Dependencies and 4NF Decomposition

```
mysql> CREATE TABLE TEACHER (
->   TID INT,
->   Subject VARCHAR(50),
->   Hobby VARCHAR(50)
-> );
Query OK, 0 rows affected (0.03 sec)

mysql> INSERT INTO TEACHER
-> VALUES
-> (101, 'DBMS', 'Reading'),
-> (101, 'DBMS', 'Music'),
-> (101, 'Java', 'Reading'),
-> (101, 'Java', 'Music'),
-> (102, 'Python', 'Cricket'),
-> (102, 'Python', 'Reading');
Query OK, 6 rows affected (0.01 sec)
Records: 6 Duplicates: 0 Warnings: 0
```

```
mysql> SELECT * FROM TEACHER;
+----+-----+-----+
| TID | Subject | Hobby |
+----+-----+-----+
| 101 | DBMS    | Reading |
| 101 | DBMS    | Music   |
| 101 | Java    | Reading |
| 101 | Java    | Music   |
| 102 | Python  | Cricket |
| 102 | Python  | Reading |
+----+-----+-----+
```

```
mysql> CREATE TABLE TEACHER_SUBJECT (
->   TID INT,
->   Subject VARCHAR(50),
->   PRIMARY KEY (TID, Subject)
-> );
Query OK, 0 rows affected (0.03 sec)

mysql> CREATE TABLE TEACHER_HOBBY (
->   TID INT,
->   Hobby VARCHAR(50),
->   PRIMARY KEY (TID, Hobby)
-> );
Query OK, 0 rows affected (0.02 sec)

mysql> INSERT INTO TEACHER_SUBJECT
-> VALUES
-> (101, 'DBMS'),
-> (101, 'Java'),
-> (102, 'Python');
Query OK, 3 rows affected (0.01 sec)
Records: 3 Duplicates: 0 Warnings: 0
```

```
mysql> SELECT * FROM TEACHER_SUBJECT;
+----+-----+
| TID | Subject |
+----+-----+
| 101 | DBMS    |
| 101 | Java    |
| 102 | Python  |
+----+-----+
3 rows in set (0.00 sec)

mysql> INSERT INTO TEACHER_HOBBY
-> VALUES
-> (101, 'Reading'),
-> (101, 'Music'),
-> (102, 'Cricket'),
-> (102, 'Reading');
Query OK, 4 rows affected (0.01 sec)
Records: 4 Duplicates: 0 Warnings: 0

mysql> SELECT * FROM TEACHER_HOBBY;
+----+-----+
| TID | Hobby |
+----+-----+
| 101 | Music |
| 101 | Reading |
| 102 | Cricket |
| 102 | Reading |
+----+-----+
4 rows in set (0.00 sec)
```

```
mysql> SELECT
->     TS.TID,
->     TS.Subject,
->     TH.Hobby
-> FROM TEACHER_SUBJECT TS
-> JOIN TEACHER_HOBBY TH
-> ON TS.TID = TH.TID;
+-----+-----+-----+
| TID | Subject | Hobby |
+-----+-----+-----+
| 101 | DBMS    | Music |
| 101 | DBMS    | Reading |
| 101 | Java    | Music |
| 101 | Java    | Reading |
| 102 | Python  | Cricket |
| 102 | Python  | Reading |
+-----+-----+-----+
6 rows in set (0.00 sec)
```

Final Decomposition

The original relation:

TEACHER(TID,Subject,Hobby)

is decomposed into:

TEACHER_SUBJECT(TID,Subject)

and

TEACHER_HOBBY(TID,Hobby)

Conclusion

The original TEACHER relation contains two independent multi-valued facts: **subjects taught by a teacher** and **hobbies of a teacher**. This produces redundant combinations of subjects and hobbies and violates 4NF. By decomposing it into TEACHER_SUBJECT and TEACHER_HOBBY, the multi-valued dependencies are separated and the resulting relations satisfy **Fourth Normal Form (4NF)**.

8. Demonstrate how join dependencies lead to redundancy. Decompose a given relation to 5NF (Project-Join Normal Form).

5NF Decomposition Using Join Dependencies

1. Introduction

Fifth Normal Form (5NF), also called **Project-Join Normal Form (PJ/NF)**, deals with **join dependencies**.

A relation is in **5NF** when every non-trivial join dependency is implied by the candidate keys of the relation.

Join dependencies can cause **redundancy** when a relation stores several independent many-to-many relationships in a single table.

```
mysql> CREATE TABLE SPJ (  
-> Supplier VARCHAR(20),  
-> Part VARCHAR(20),  
-> Project VARCHAR(20)  
-> );  
Query OK, 0 rows affected (0.03 sec)  
  
mysql> INSERT INTO SPJ  
-> VALUES  
-> ('S1', 'P1', 'J1'),  
-> ('S1', 'P1', 'J2'),  
-> ('S1', 'P2', 'J1'),  
-> ('S1', 'P2', 'J2');  
Query OK, 4 rows affected (0.01 sec)  
Records: 4 Duplicates: 0 Warnings: 0  
  
mysql> SELECT * FROM SPJ;  
+-----+-----+-----+  
| Supplier | Part | Project |  
+-----+-----+-----+  
| S1       | P1   | J1      |  
| S1       | P1   | J2      |  
| S1       | P2   | J1      |  
| S1       | P2   | J2      |  
+-----+-----+-----+  
4 rows in set (0.00 sec)
```

```
mysql> CREATE TABLE SUPPLIER_PART (  
-> Supplier VARCHAR(20),  
-> Part VARCHAR(20),  
-> PRIMARY KEY (Supplier, Part)  
-> );  
Query OK, 0 rows affected (0.03 sec)  
  
mysql> INSERT INTO SUPPLIER_PART  
-> VALUES  
-> ('S1', 'P1'),  
-> ('S1', 'P2');  
Query OK, 2 rows affected (0.01 sec)  
Records: 2 Duplicates: 0 Warnings: 0  
  
mysql> SELECT * FROM SUPPLIER_PART;  
+-----+-----+  
| Supplier | Part |  
+-----+-----+  
| S1       | P1   |  
| S1       | P2   |  
+-----+-----+  
2 rows in set (0.00 sec)  
  
mysql> CREATE TABLE SUPPLIER_PROJECT (  
-> Supplier VARCHAR(20),  
-> Project VARCHAR(20),  
-> PRIMARY KEY (Supplier, Project)  
-> );  
Query OK, 0 rows affected (0.03 sec)
```

```
mysql> INSERT INTO SUPPLIER_PROJECT
-> VALUES
-> ('S1', 'J1'),
-> ('S1', 'J2');
Query OK, 2 rows affected (0.01 sec)
Records: 2 Duplicates: 0 Warnings: 0
```

```
mysql> SELECT * FROM SUPPLIER_PROJECT;
+-----+-----+
| Supplier | Project |
+-----+-----+
| S1       | J1      |
| S1       | J2      |
+-----+-----+
2 rows in set (0.00 sec)
```

```
mysql> CREATE TABLE PART_PROJECT (
-> Part VARCHAR(20),
-> Project VARCHAR(20),
-> PRIMARY KEY (Part, Project)
-> );
Query OK, 0 rows affected (0.03 sec)
```

```
mysql> INSERT INTO PART_PROJECT
-> VALUES
-> ('P1', 'J1'),
-> ('P1', 'J2'),
-> ('P2', 'J1'),
-> ('P2', 'J2');
```

```
mysql> SELECT * FROM PART_PROJECT;
+-----+-----+
| Part | Project |
+-----+-----+
| P1   | J1      |
| P1   | J2      |
| P2   | J1      |
| P2   | J2      |
+-----+-----+
4 rows in set (0.00 sec)
```

```
mysql> SELECT
-> SP.Supplier,
-> SP.Part,
-> PP.Project
-> FROM SUPPLIER_PART SP
-> JOIN PART_PROJECT PP
-> ON SP.Part = PP.Part
-> JOIN SUPPLIER_PROJECT SJ
-> ON SP.Supplier = SJ.Supplier
-> AND PP.Project = SJ.Project;
```

```
+-----+-----+-----+
| Supplier | Part | Project |
+-----+-----+-----+
| S1       | P1   | J1      |
| S1       | P1   | J2      |
| S1       | P2   | J1      |
| S1       | P2   | J2      |
+-----+-----+-----+
```

8. Advantages of 5NF Decomposition

1. **Reduces redundancy** by separating independent relationships.
2. **Avoids update anomalies** because information is stored only once.
3. **Avoids insertion anomalies** because new relationships can be added independently.
4. **Avoids deletion anomalies** because deleting one relationship does not unnecessarily remove another.
5. **Improves database consistency.**
6. Allows the original relation to be reconstructed through joins.

Final Answer

The original relation:

SPJ(Supplier,Part,Project)

is decomposed into:

SP(Supplier,Part) SJ(Supplier,Project) PJ(Part,Project)

with the join dependency:

*(SP,SJ,PJ)

Thus, join dependencies can produce redundancy when independent relationships are stored together. Decomposing the relation into its projections removes this redundancy and results in a 5NF (Project-Join Normal Form) design.

9. Given a poorly designed database schema for a college, identify all anomalies (insertion, deletion, update) and normalize it to 3NF.

Normalizing a Poorly Designed College Database to 3NF

1. Introduction

Consider a poorly designed college database where student, department, course, instructor, and enrollment information are stored in a **single relation**.

Suppose the relation is:

COLLEGE(StudentID,StudentName,DeptID,DeptName,CourseID,CourseName,InstructorID,InstructorName,Grade)

This design contains redundant information and can lead to **insertion, deletion, and update anomalies**.

```
mysql> CREATE TABLE COLLEGE (
  -> StudentID INT,
  -> StudentName VARCHAR(50),
  -> DeptID INT,
  -> DeptName VARCHAR(50),
  -> CourseID INT,
  -> CourseName VARCHAR(50),
  -> InstructorID INT,
  -> InstructorName VARCHAR(50),
  -> Grade VARCHAR(5)
  -> );
Query OK, 0 rows affected (0.05 sec)

mysql> INSERT INTO COLLEGE
  -> VALUES
  -> (101, 'Arun', 10, 'CSE', 201, 'DBMS', 501, 'Kumar', 'A'),
  -> (101, 'Arun', 10, 'CSE', 202, 'Java', 502, 'Priya', 'B'),
  -> (102, 'Ravi', 10, 'CSE', 201, 'DBMS', 501, 'Kumar', 'A'),
  -> (103, 'Meena', 20, 'ECE', 203, 'Networks', 503, 'Raj', 'A'),
  -> (104, 'Anu', 20, 'ECE', 204, 'Python', 504, 'Divya', 'B');
Query OK, 5 rows affected (0.01 sec)
Records: 5 Duplicates: 0 Warnings: 0

mysql> SELECT * FROM COLLEGE;
```

StudentID	StudentName	DeptID	DeptName	CourseID	CourseName	InstructorID	InstructorName	Grade
101	Arun	10	CSE	201	DBMS	501	Kumar	A
101	Arun	10	CSE	202	Java	502	Priya	B
102	Ravi	10	CSE	201	DBMS	501	Kumar	A
103	Meena	20	ECE	203	Networks	503	Raj	A
104	Anu	20	ECE	204	Python	504	Divya	B

```
mysql> UPDATE COLLEGE
  -> SET DeptName = 'Computer Science and Engineering'
  -> WHERE DeptID = 10;
Query OK, 3 rows affected (0.01 sec)
Rows matched: 3 Changed: 3 Warnings: 0

mysql> DELETE FROM COLLEGE
  -> WHERE StudentID = 104;
Query OK, 1 row affected (0.01 sec)

mysql> CREATE TABLE STUDENT (
  -> StudentID INT PRIMARY KEY,
  -> StudentName VARCHAR(50),
  -> DeptID INT
  -> );
Query OK, 0 rows affected (0.03 sec)

mysql> INSERT INTO STUDENT
  -> VALUES
  -> (101, 'Arun', 10),
  -> (102, 'Ravi', 10),
  -> (103, 'Meena', 20),
  -> (104, 'Anu', 20);
Query OK, 4 rows affected (0.01 sec)
Records: 4 Duplicates: 0 Warnings: 0
```

```
mysql> SELECT * FROM STUDENT;
+-----+-----+-----+
| StudentID | StudentName | DeptID |
+-----+-----+-----+
| 101 | Arun | 10 |
| 102 | Ravi | 10 |
| 103 | Meena | 20 |
| 104 | Anu | 20 |
+-----+-----+-----+
4 rows in set (0.00 sec)

mysql> CREATE TABLE COURSE (
-> CourseID INT PRIMARY KEY,
-> CourseName VARCHAR(50),
-> InstructorID INT
-> );
Query OK, 0 rows affected (0.03 sec)

mysql> INSERT INTO COURSE
-> VALUES
-> (201, 'DBMS', 501),
-> (202, 'Java', 502),
-> (203, 'Networks', 503),
-> (204, 'Python', 504);
Query OK, 4 rows affected (0.01 sec)
Records: 4 Duplicates: 0 Warnings: 0

mysql> CREATE TABLE ENROLLMENT (
-> StudentID INT,
-> CourseID INT,
-> Grade VARCHAR(5),
-> PRIMARY KEY (StudentID, CourseID)
```

```
-> PRIMARY KEY (StudentID, CourseID)
-> );
Query OK, 0 rows affected (0.03 sec)

mysql> INSERT INTO ENROLLMENT
-> VALUES
-> (101, 201, 'A'),
-> (101, 202, 'B'),
-> (102, 201, 'A'),
-> (103, 203, 'A'),
-> (104, 204, 'B');
Query OK, 5 rows affected (0.01 sec)
Records: 5 Duplicates: 0 Warnings: 0
```

```
mysql> CREATE TABLE DEPARTMENT (
-> DeptID INT PRIMARY KEY,
-> DeptName VARCHAR(50)
-> );
Query OK, 0 rows affected (0.04 sec)

mysql> INSERT INTO DEPARTMENT
-> VALUES
-> (10, 'CSE'),
-> (20, 'ECE');
Query OK, 2 rows affected (0.01 sec)
Records: 2 Duplicates: 0 Warnings: 0

mysql> CREATE TABLE INSTRUCTOR (
-> InstructorID INT PRIMARY KEY,
-> InstructorName VARCHAR(50)
-> );
Query OK, 0 rows affected (0.03 sec)

mysql> INSERT INTO INSTRUCTOR
-> VALUES
-> (501, 'Humar'),
-> (502, 'Priya'),
-> (503, 'Raj'),
-> (504, 'Divya');
Query OK, 4 rows affected (0.01 sec)
Records: 4 Duplicates: 0 Warnings: 0

mysql>
mysql> ALTER TABLE STUDENT
-> ADD FOREIGN KEY (DeptID)
-> REFERENCES DEPARTMENT(DeptID);
```

```
mysql> ALTER TABLE STUDENT
-> ADD FOREIGN KEY (DeptID)
-> REFERENCES DEPARTMENT(DeptID);
Query OK, 4 rows affected (0.12 sec)
Records: 4 Duplicates: 0 Warnings: 0

mysql> ALTER TABLE COURSE
-> ADD FOREIGN KEY (InstructorID)
-> REFERENCES INSTRUCTOR(InstructorID);
Query OK, 4 rows affected (0.11 sec)
Records: 4 Duplicates: 0 Warnings: 0

mysql> ALTER TABLE ENROLLMENT
-> ADD FOREIGN KEY (StudentID)
-> REFERENCES STUDENT(StudentID);
Query OK, 5 rows affected (0.07 sec)
Records: 5 Duplicates: 0 Warnings: 0

mysql> ALTER TABLE ENROLLMENT
-> ADD FOREIGN KEY (CourseID)
-> REFERENCES COURSE(CourseID);
Query OK, 5 rows affected (0.13 sec)
Records: 5 Duplicates: 0 Warnings: 0

mysql> SELECT
-> S.StudentID,
-> S.StudentName,
-> D.DeptName,
-> C.CourseID,
-> C.CourseName,
-> I.InstructorName,
-> E.Grade
```

```

-> I.InstructorName,
-> E.Grade
-> FROM ENROLLMENT E
-> JOIN STUDENT S
-> ON E.StudentID = S.StudentID
-> JOIN DEPARTMENT D
-> ON S.DeptID = D.DeptID
-> JOIN COURSE C
-> ON E.CourseID = C.CourseID
-> JOIN INSTRUCTOR I
-> ON C.InstructorID = I.InstructorID;

```

StudentID	StudentName	DeptName	CourseID	CourseName	InstructorName	Grade
101	Arun	CSE	201	DBMS	Kumar	A
101	Arun	CSE	202	Java	Priya	B
102	Ravi	CSE	201	DBMS	Kumar	A
103	Meena	ECE	203	Networks	Raj	A
104	Anu	ECE	204	Python	Divya	B

5 rows in set (0.01 sec)

```

mysql> UPDATE DEPARTMENT
-> SET DeptName = 'Computer Science'
-> WHERE DeptID = 10;
Query OK, 1 row affected (0.01 sec)
Rows matched: 1 Changed: 1 Warnings: 0

```

```

mysql> UPDATE DEPARTMENT
-> SET DeptName = 'Computer Science'
-> WHERE DeptID = 10;
Query OK, 1 row affected (0.01 sec)
Rows matched: 1 Changed: 1 Warnings: 0

mysql> select * from department;

```

DeptID	DeptName
10	Computer Science
20	ECE

2 rows in set (0.00 sec)

Relationship Structure

DEPARTMENT

| 1 : N

STUDENT

| 1 : N

ENROLLMENT

| N : 1

COURSE

| N : 1

INSTRUCTOR

12. Summary of Normalization

Normal Form	Problem Removed	Result
UNF	Repeating/unstructured data	Poorly designed single table
1NF	Non-atomic/repeating values	Atomic values
2NF	Partial dependencies	Student, Course, Enrollment separated
3NF	Transitive dependencies	Department and Instructor separated

Final Conclusion

The original college relation contains **insertion, deletion, and update anomalies** because student, department, course, instructor, and enrollment information are mixed in one table. The candidate key is (StudentID, CourseID), and partial dependencies prevent the relation from being in 2NF. After removing partial dependencies and then removing transitive dependencies, the database is decomposed into **STUDENT, DEPARTMENT, COURSE, INSTRUCTOR, and ENROLLMENT** relations. These relations are in 3NF, reduce redundancy, and prevent insertion, deletion, and update anomalies.

10. Compute the closure of attribute sets for the given FDs and determine all candidate keys for the relation.

Attribute Closure and Candidate Keys

To demonstrate the process, consider the relation:

$R(A,B,C,D,E)$

with the functional dependencies:

$F=\{A \rightarrow BC, BC \rightarrow D, D \rightarrow E\}$

We need to **compute attribute closures** and determine the **candidate key(s)**.

1. What is Attribute Closure?

The **closure of an attribute set X**, written as X^+ , is the set of all attributes that can be functionally determined from X using the given functional dependencies.

It is mainly used to:

- Find whether an attribute set is a **superkey**.
- Find **candidate keys**.
- Check functional dependencies.

2. Find A^+

Start with:

$A^+=\{A\}$

Using:

$A \rightarrow BC$

add B and C:

$A^+=\{A,B,C\}$

Now we have B and C, so apply:

$BC \rightarrow D$

Add D:

$A^+ = \{A, B, C, D\}$

Now apply:

$D \rightarrow E$

Add E:

$A^+ = \{A, B, C, D, E\}$

Since A^+ contains **all attributes of R**, A is a **superkey**.

Because A itself is sufficient, it is also a **candidate key**.

Candidate Key=A

3. Find B^+

Start:

$B^+ = \{B\}$

There is no FD beginning with B alone.

Therefore:

$B^+ = \{B\}$

B is not a superkey.

4. Find C^+

Start:

$C^+ = \{C\}$

There is no FD beginning with C alone.

Therefore:

$C^+ = \{C\}$

C is not a superkey.

5. Find D^+

Start:

$D^+ = \{D\}$

Using:

$D \rightarrow E$

we get:

$D^+ = \{D, E\}$

Therefore:

$D^+ = \{D, E\}$

D is not a superkey.

6. Find E^+

Start:

$E^+ = \{E\}$

There is no FD beginning with E.

Therefore:

$E^+ = \{E\}$

E is not a superkey.

7. Check Other Attribute Combinations

Since A alone is already a candidate key, any set containing A is **not minimal**.

For example:

$(AB)^+ = \{A, B, C, D, E\}$

but AB is not a candidate key because A alone already determines everything.

Similarly:

$(AC)^+ = \{A, B, C, D, E\}$

but AC is not minimal.

We therefore need to check combinations **not containing A**.

Check BC

Start:

$BC^+ = \{B, C\}$

Using:

$BC \rightarrow D$

we get:

$BC^+ = \{B, C, D\}$

Using:

$D \rightarrow E$

we get:

$BC^+ = \{B, C, D, E\}$

A is still missing.

Therefore:

BC is not a superkey

Check BD

Start:

$BD^+ = \{B, D\}$

Using:

$D \rightarrow E$

we get:

$BD^+ = \{B, D, E\}$

A and C are missing.

So BD is not a superkey.

Check CD

Start:

$CD^+ = \{C, D\}$

Using:

$D \rightarrow E$

we get:

$CD^+ = \{C, D, E\}$

A and B are missing.

So CD is not a superkey.

Thus, there are no additional candidate keys.

8. Closure Summary

Attribute Set	Closure	Superkey?
A	{A, B, C, D, E}	Yes
B	{B}	No
C	{C}	No
D	{D, E}	No
E	{E}	No
BC	{B, C, D, E}	No
BD	{B, D, E}	No
CD	{C, D, E}	No

Final Answer

The complete attribute closure is:

$A^+ = \{A, B, C, D, E\}$

Therefore, the only candidate key is:

Candidate Key = {A}

Reason: A alone determines every attribute in the relation, and no proper subset of A exists. Hence, A is a minimal superkey and therefore the **only candidate key**.